Notes: Gu, Kelly, and Xiu (RFS, 2020)

Empirical Asset Pricing via Machine Learning

Contents

1 Big picture

- **Question.** How much can machine learning improve the measurement of conditional expected stock returns, relative to standard asset-pricing regressions and portfolio-sorting methods?
- **Setting.** The paper studies U.S. equity excess returns from **1957–2016**, using a very large predictor set: **94 stock characteristics**, their interactions with **8 macroeconomic variables**, and **74 industry dummies**, for a total of **920 baseline features**.
- **Core challenge.** Asset-return prediction is a very low signal-to-noise problem. The literature has accumulated hundreds of candidate predictors that are highly correlated, often noisy, and may matter through nonlinearities and interactions rather than simple additive linear effects.
- **Main methodological contribution.** The paper provides a comparative "horse race" among the canonical machine-learning methods used for prediction: OLS, penalized linear models, dimension reduction, generalized linear models with splines, tree methods, and neural networks, all evaluated in a strict out-of-sample design.
- **Main stock-level result.** Standard OLS with all 920 features performs disastrously out of sample, with monthly panel $R^2_{oos} = -3.46\%$. In contrast, machine-learning methods perform much better: elastic net recovers positive predictive power, PCR/PLS improve further, and the best methods are **trees and neural networks**, with stock-level monthly R^2_{oos} around **0.33%–0.40%**.
- **Main portfolio-level result.** Bottom-up portfolio forecasts show even larger gains. For the S&P 500, a three-layer neural network (NN3) achieves monthly out-of-sample R^2 of about **1.80%**, versus **-0.22%** for the benchmark OLS-3 model.
- **Economic value.** An investor who times the S&P 500 using neural-network forecasts raises the annualized Sharpe ratio from about **0.51** (buy-and-hold) to about **0.77**. A value-weighted long–short decile spread formed on neural-network stock forecasts reaches an annualized Sharpe ratio around **1.35**, versus roughly **0.61** for the benchmark regression strategy.
- **What drives the gains.** The paper's central interpretation is that the big gains come mainly from allowing **nonlinear interactions** among predictors. Univariate nonlinearities alone help less. This is why generalized linear spline models improve only modestly, while trees and neural networks dominate.
- **Which signals matter most.** Across methods, the strongest predictors are consistently variations of **price trends** (momentum, reversal, industry momentum), **liquidity** (market value, dollar volume, bid-ask spread, illiquidity), and **volatility/risk** (return volatility, idiosyncratic volatility, beta).
- **Economic takeaway.** Machine learning materially improves the measurement of expected returns, but these forecasts are still *measurements*, not structural explanations. The paper is about better risk-premium estimation, not about proving a particular economic mechanism behind returns.

2 Data and measurement (key objects)

2.1 Returns sample and basic outcome variable

- **Return data.** Monthly total stock returns come from **CRSP** for all firms listed on the **NYSE, AMEX, and NASDAQ**.
- **Sample period.** The sample starts in **March 1957** (the start of the S&P 500) and ends in **December 2016**, giving roughly **60 years** of monthly data.
- **Sample size.** The dataset contains nearly **30,000 stocks** overall, with an average of more than **6,200 stocks per month**.
- **Outcome variable.** The object being predicted is the next-period excess return:

  $$r_{i,t+1} = R_{i,t+1} - r_t^f,$$

  where r_t^f is proxied by the Treasury-bill rate.
- **Inclusive sample design.** The authors deliberately keep a broad asset universe, including stocks below $5, financial firms, and stocks outside the narrow share-code filters often used in asset-pricing work. The idea is to avoid selection bias, preserve important S&P 500 constituents, and increase the observation count relative to the parameter count.

2.2 Predictors: characteristics, macro variables, and industry dummies

- **Stock characteristics.** The paper builds **94 stock-level characteristics** based on the cross-section of returns literature.
- **Update frequencies.**
  - **61** are updated annually,
  - **13** are updated quarterly,
  - **20** are updated monthly.
- **Industry controls.** The authors add **74** two-digit SIC industry dummies.
- **Macroeconomic predictors.** They use the eight Welch–Goyal style variables:
  - dividend-price ratio (dp),
  - earnings-price ratio (ep),
  - aggregate book-to-market (bm),
  - net equity expansion ($ntis$),
  - Treasury-bill rate (tbl),
  - term spread (tms),
  - default spread (dfy),
  - stock-market variance ($svar$).

2.3 Feature engineering and timing

- **Feature vector.** The baseline covariate vector is

$$z_{i,t} = x_t \otimes c_{i,t},$$

  where $c_{i,t}$ is the vector of stock characteristics and x_t contains a constant plus the eight macro variables.

- **Total feature count.** Since there are 94 characteristics and $8 + 1$ macro states including the constant, the interacted characteristic block contributes $94 \times 9 = 846$ features. Adding 74 industry dummies yields **920 total baseline predictors**.

- **Cross-sectional normalization.** Characteristics are ranked cross-sectionally each month and mapped into the $[-1, 1]$ interval.

- **Publication-lag discipline.** To avoid look-ahead bias, the authors lag information availability:
  - monthly variables by at most **1 month**,
  - quarterly variables by at least **4 months**,
  - annual variables by at least **6 months**.

- **Missing-data treatment.** Missing characteristics are replaced with the cross-sectional median for that characteristic in that month.

2.4 Train/validation/test split and recursive estimation

- **Training sample. 1957–1974** (18 years).
- **Validation sample. 1975–1986** (12 years).
- **Out-of-sample testing sample. 1987–2016** (30 years).
- **Recursive refitting.** Models are re-estimated **once per year**, not every month. Each re-estimation expands the training sample by one year and rolls the validation window forward to keep its length fixed.
- **Why yearly refits.** Machine-learning methods are computationally intensive, and many accounting signals update only annually or quarterly, so yearly re-estimation captures most of the relevant signal refresh.

2.5 Portfolio objects used for economic evaluation

- **Prespecified portfolios.** The paper aggregates stock-level forecasts into:
  - the **S&P 500**,
  - the Fama–French **SMB, HML, RMW, CMA, UMD** factors,
  - and **24 long-only subcomponent portfolios** used to build those factors.
- **Machine-learning decile portfolios.** At each month-end, stocks are sorted into deciles based on predicted next-month returns, and the paper studies long–short $10 - 1$ spreads.
- **Risk-adjusted evaluation.** Economic performance is assessed with Sharpe ratios, drawdowns, turnover, and alphas relative to the Fama–French five-factor model plus momentum.

3 Models and empirical specifications (all equations + interpretation)

3.1 Overarching prediction problem

The paper starts from a very general additive representation of excess returns:

$$r_{i,t+1} = E_t(r_{i,t+1}) + \varepsilon_{i,t+1}, \tag{1}$$
$$E_t(r_{i,t+1}) = g^*(z_{i,t}). \tag{2}$$

Here:

- $r_{i,t+1}$ is the next-month excess return on stock i,
- $z_{i,t}$ is the predictor vector,
- $g^*(\cdot)$ is an unknown conditional expectation function.

Interpretation. The entire paper is about approximating $g^*(z_{i,t})$ as well as possible out of sample. The issue is not whether expected returns exist, but how to estimate them when the conditioning information set is high-dimensional and potentially nonlinear.

3.2 Sample splitting, tuning, and the logic of machine learning

- Each method has **hyperparameters**—for example, penalty strength, tree depth, number of principal components, or network architecture.
- Hyperparameters are selected on a **validation sample**, not on the final test sample.
- The ultimate performance metric is computed on a **testing sample** that is used for neither estimation nor tuning.

Interpretation. This is central. In the paper's design, machine learning is not "flexibility without discipline." It is high flexibility combined with aggressive regularization and honest out-of-sample evaluation.

3.3 Simple linear model, weighted loss, and robust Huber loss

The simplest model is linear:

$$g(z_{i,t};\theta) = z'_{i,t}\theta. \tag{3}$$

The baseline objective is pooled least squares:

$$\mathcal{L}(\theta) = \frac{1}{NT}\sum_{i=1}^{N}\sum_{t=1}^{T}\big(r_{i,t+1} - g(z_{i,t};\theta)\big)^2. \tag{4}$$

A weighted least-squares version is:

$$\mathcal{L}_W(\theta) = \frac{1}{NT}\sum_{i=1}^{N}\sum_{t=1}^{T}w_{i,t}\big(r_{i,t+1} - g(z_{i,t};\theta)\big)^2. \tag{5}$$

To reduce the influence of heavy-tailed outliers, the paper also studies Huber loss:

$$\mathcal{L}_H(\theta) = \frac{1}{NT}\sum_{i=1}^{N}\sum_{t=1}^{T} H(r_{i,t+1} - g(z_{i,t};\theta), \xi), \tag{6}$$

where

$$H(x;\xi) = \begin{cases} x^2, & |x| \le \xi, \\ 2\xi|x| - \xi^2, & |x| > \xi. \end{cases}$$

Interpretation. OLS treats every squared error symmetrically and is fragile in a high-dimensional, heavy-tailed environment. Huber loss preserves least-squares behavior for moderate errors but downweights extreme observations, which is appealing in stock-return data.

3.4 Penalized linear model: elastic net

Penalized regression keeps the same linear functional form but adds a regularization term:

$$\mathcal{L}(\theta;\cdot) = \mathcal{L}(\theta) + \phi(\theta;\cdot). \tag{7}$$

For the elastic net,

$$\phi(\theta;\lambda,\rho) = \lambda(1-\rho)\sum_{j=1}^{P}|\theta_j| + \frac{1}{2}\lambda\rho\sum_{j=1}^{P}\theta_j^2. \tag{8}$$

- $\rho = 0$ gives the lasso (variable selection),
- $\rho = 1$ gives ridge regression (shrinkage),
- intermediate ρ combines both.

Interpretation. Elastic net is the simplest way to make the 920-feature regression viable. It reduces degrees of freedom by shrinking and zeroing out coefficients. In the paper, it rescues prediction from the large-feature OLS failure, but it still does not match the best nonlinear methods.

3.5 Dimension reduction: PCR and PLS

The vectorized regression is written as

$$R = Z\theta + E, \tag{9}$$

and the dimension-reduced version is

$$R = (Z\Omega_K)\theta_K + \widetilde{E}. \tag{10}$$

PCR. Principal components regression chooses the j-th component as

$$w_j = \arg\max_{w} \operatorname{Var}(Zw) \quad \text{s.t. } w'w = 1,\ \operatorname{Cov}(Zw, Zw_\ell) = 0 \text{ for } \ell < j. \tag{11}$$

PLS. Partial least squares instead chooses the j-th component as

$$w_j = \arg\max_w \mathrm{Cov}^2(R, Zw) \quad \text{s.t. } w'w = 1, \ \mathrm{Cov}(Zw, Zw_\ell) = 0 \text{ for } \ell < j. \tag{12}$$

Interpretation.

- **PCR** compresses predictors based on how much common variation among predictors they explain.
- **PLS** compresses predictors based on how much predictive covariance they have with returns.

Both are ways of saying: "Instead of estimating hundreds of noisy coefficients, project the data onto a handful of informative directions first." In the paper, this works well and clearly outperforms unrestricted OLS.

3.6 Generalized linear model with splines and group lasso

The generalized linear model expands each predictor with spline basis terms:

$$g(z; \theta, p(\cdot)) = \sum_{j=1}^{P} p(z_j)'\theta_j, \tag{13}$$

where $p(\cdot)$ is a spline basis.

Because spline expansion multiplies the number of parameters, the paper regularizes with the **group lasso**:

$$\phi(\theta; \lambda, K) = \lambda \sum_{j=1}^{P} \left(\sum_{k=1}^{K} \theta_{j,k}^2 \right)^{1/2}. \tag{14}$$

Interpretation. This model allows *univariate* nonlinearities, such as a curved size effect or a kinked momentum effect, but it still adds predictors one by one. It does not naturally handle the huge space of multiway interactions. That is why it improves modestly, but far less than tree methods and neural networks.

3.7 Regression trees, boosting, and random forests

A regression tree partitions the predictor space into rectangular regions and assigns a constant fit in each region:

$$g(z_{i,t}; \theta, K, L) = \sum_{k=1}^{K} \theta_k \mathbf{1}\{z_{i,t} \in C_k(L)\}. \tag{15}$$

For a candidate partition C, the impurity criterion is

$$\mathcal{H}(\theta, C) = \frac{1}{|C|} \sum_{z_{i,t} \in C} (r_{i,t+1} - \theta)^2. \tag{16}$$

- **Random forest** averages many bagged trees and uses random predictor subsets at each split to decorrelate trees.

- **Gradient-boosted regression trees (GBRT)** sequentially fits shallow trees to the residuals of previous trees and adds them with shrinkage.

Interpretation. Trees are powerful precisely because they make interactions cheap to represent. A split on size followed by a split on momentum is already a size–momentum interaction. A depth-L tree can naturally encode $(L-1)$-way interactions without writing them out explicitly.

3.8 Neural networks

For a feed-forward neural network, the output of neuron k in hidden layer ℓ is

$$x_k^{(\ell)} = \mathrm{ReLU}\Big((x^{(\ell-1)})'\theta_k^{(\ell-1)}\Big), \tag{17}$$

with final output

$$g(z;\theta) = (x^{(L-1)})'\theta^{(L-1)}. \tag{18}$$

The paper uses the rectified linear unit (ReLU):

$$\mathrm{ReLU}(x) = \begin{cases} 0, & x < 0, \\ x, & x \geq 0. \end{cases}$$

Network architectures are fixed ex ante and compared directly:

- NN1: one hidden layer with 32 neurons,
- NN2: layers $32, 16$,
- NN3: layers $32, 16, 8$,
- NN4: layers $32, 16, 8, 4$,
- NN5: layers $32, 16, 8, 4, 2$.

Estimation uses stochastic gradient descent with several regularizers:

- ℓ_1 penalization,
- learning-rate shrinkage,
- early stopping,
- batch normalization,
- and ensemble averaging over random initializations.

Interpretation. Neural networks are the most flexible method in the paper. The main empirical lesson is interesting: some depth helps, but very deep networks do not. In asset pricing, the data are not large enough and the signal is not strong enough to make very deep learning outperform moderately deep, "shallow" networks.

3.9 Performance evaluation and model comparison

The paper's main stock-level metric is the pooled out-of-sample R^2:

$$R^2_{oos} = 1 - \frac{\sum_{(i,t)\in T_3}(r_{i,t+1} - \hat{r}_{i,t+1})^2}{\sum_{(i,t)\in T_3} r^2_{i,t+1}}. \tag{19}$$

Important detail. The denominator does *not* subtract the historical mean. The benchmark is a zero forecast, not the historical average excess return. This is crucial because at the individual-stock level the historical mean is itself an extremely noisy forecast.

To compare methods, the paper adapts the Diebold–Mariano test to panel data by first averaging squared error differences across stocks each month:

$$d_{12,t+1} = \frac{1}{n_{3,t+1}} \sum_{i=1}^{n_{3,t+1}} \left((\hat{e}^{(1)}_{i,t+1})^2 - (\hat{e}^{(2)}_{i,t+1})^2 \right). \tag{20}$$

Interpretation. This turns the comparison problem into a time-series test of monthly average error differences, which is more sensible than pretending all stock-level errors are cross-sectionally independent.

3.10 Variable importance, marginal effects, and interaction plots

The paper uses two main importance measures:

- the drop in predictive R^2 when a predictor is shut down,
- and a sensitivity measure based on sums of squared partial derivatives (or mean decrease in impurity for trees).

It also studies:

- **marginal association plots**, holding other variables fixed at medians,
- and **interaction plots**, especially for the NN3 model.

Interpretation. These tools help open the "black box." They do not turn the exercise into causal identification, but they do show where predictive gains come from and what nonlinearities the model is picking up.

3.11 Bottom-up portfolio forecasts and machine-learning portfolios

For a portfolio p with known weights $w^p_{i,t}$, the bottom-up portfolio forecast is

$$\hat{r}^p_{t+1} = \sum_{i=1}^{n} w^p_{i,t}\hat{r}_{i,t+1}.$$

Campbell–Thompson's mean-variance mapping implies that a predictive R^2 translates into a Sharpe ratio

$$SR^* = \sqrt{SR^2 + \frac{R^2}{1-R^2}}.$$

The paper also studies direct long–short decile portfolios based on predicted stock returns.

Interpretation. This is how the paper turns statistical fit into economic significance. A tiny improvement in monthly R^2 can matter economically if it maps into better market timing or better long–short sorting performance.

4 Identification and empirical credibility (what makes the results persuasive)

4.1 What is hard in this problem

- **Signal-to-noise is tiny.** Most return variation is unexpected news, not predictable expected return.
- **Predictor dimension is huge.** There are hundreds of candidate signals, many of them highly correlated or redundant.
- **Functional form is unknown.** It is unclear whether returns depend on predictors linearly, nonlinearly, or through interactions.
- **Standard regression is fragile.** With many predictors, unrestricted OLS becomes unstable and overfits in-sample noise.

4.2 Why the design is credible for prediction

- **Strict time ordering.** Training, validation, and testing samples respect chronology.
- **True out-of-sample testing.** The test period is never used for parameter estimation or tuning.
- **Common data environment.** All methods are fed the same basic information set, so differences in performance reflect model architecture rather than informational advantages.
- **Recursive re-estimation.** Models are updated as data accumulate, which mimics real-world forecasting practice.

4.3 What identifies the incremental gains of machine learning

- The comparison between **OLS with 920 variables** and **elastic net / PCR / PLS** isolates the value of regularization and dimension reduction.
- The comparison between **linear methods** and the **generalized linear spline model** isolates the value of univariate nonlinearities.
- The comparison between **generalized linear models** and **trees / neural networks** isolates the value of **interactions** and richer nonlinearities.

Interpretation. This is the most important identification logic in the paper. The fact that GLM improves only modestly, while trees and neural nets improve a lot, strongly suggests that predictor interactions—not just curvature in single predictors—are the main missing ingredient in traditional asset-pricing regressions.

4.4 Why the variable-importance results are persuasive

- Different models rank predictors very similarly despite having very different functional forms.
- The dominant variables are stable across recursive samples.
- Placebo characteristics inserted into the predictor set end up near the bottom of importance rankings.

Interpretation. The paper is not just finding one quirky black-box model. Many methods point to the same economic signals, which makes the predictive message more credible.

4.5 What the paper does *not* identify

- The paper does **not** identify a structural asset-pricing model.
- It does **not** distinguish whether predictability comes from rational risk premia, behavioral mispricing, intermediary frictions, or other mechanisms.
- It does **not** prove that the most important predictors are causal drivers of expected returns.

Interpretation. The authors are explicit about this. Machine learning improves *measurement*. Economic mechanism still requires additional structure.

4.6 Relation to the literature

- Relative to classic cross-sectional work (Fama–French, Lewellen), the paper shows that a much larger predictor space can be used productively if one regularizes appropriately.
- Relative to the time-series equity premium literature (Welch–Goyal), it shows that bottom-up aggregation of stock-level forecasts can beat standard macro-only approaches.
- Relative to recent finance–ML papers, it is a broad comparative benchmark paper: not one new estimator, but a systematic mapping from method class to forecasting performance.

5 Tables and figures

5.1 Figure 1 and Figure 2: intuition for trees and neural networks

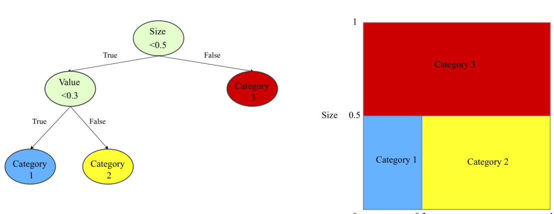


Figure 1
Regression tree example
This figure presents the diagrams of a regression tree (left) and its equivalent representation (right) in the space of two characteristics (size and value). The terminal nodes of the tree are colored in blue, yellow, and red. Based on their values of these two characteristics, the sample of individual stocks is divided into three categories.

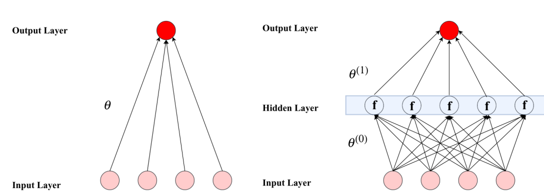


Figure 2
Neural networks
This figure provides diagrams of two simple neural networks with (right) or without (left) a hidden layer. Pink circles denote the input layer, and dark red circles denote the output layer. Each arrow is associated with a weight parameter. In the network with a hidden layer, a nonlinear activation function f transforms the inputs before passing them on to the output.

Read:

- Figure 1 gives the intuition for regression trees: partition the predictor space into regions, then predict with the average return in each region.
- Figure 2 gives the intuition for neural networks: repeated layers of weighted linear combinations plus nonlinear activation functions.
- These figures matter because the paper's main result is ultimately about **functional form**. Trees and neural networks win because they cheaply represent interactions and nonlinearities that are cumbersome in classical regressions.

5.2 Table 1, Table 2, and Figure 3: out-of-sample fit and model complexity

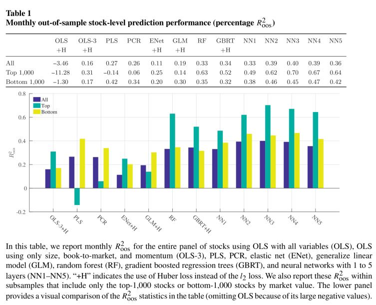


Table 1
Monthly out-of-sample stock-level prediction performance (percentage R^2_{oos})

	OLS +H	OLS-3 +H	PLS	PCR	ENet +H	GLM +H	RF	GBRT +H	NN1	NN2	NN3	NN4	NN5
All	–3.46	0.16	0.27	0.26	0.11	0.19	0.33	0.34	0.33	0.39	0.40	0.39	0.36
Top 1,000	–11.28	0.31	–0.14	0.06	0.25	0.14	0.63	0.52	0.49	0.62	0.70	0.67	0.64
Bottom 1,000	–1.30	0.17	0.42	0.34	0.20	0.30	0.35	0.32	0.38	0.46	0.45	0.47	0.42

In this table, we report monthly R^2_{oos} for the entire panel of stocks using OLS with all variables (OLS), OLS using only size, book-to-market, and momentum (OLS-3), PLS, PCR, elastic net (ENet), generalize linear model (GLM), random forest (RF), gradient boosted regression trees (GBRT), and neural networks with 1 to 5 layers (NN1–NN5). "+H" indicates the use of Huber loss instead of the l_2 loss. We also report these R^2_{oos} within subsamples that include only the top-1,000 stocks or bottom-1,000 stocks by market value. The lower panel provides a visual comparison of the R^2_{oos} statistics in the table (omitting OLS because of its large negative values).

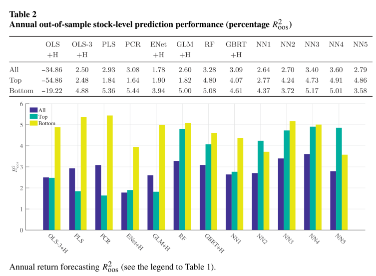


Table 2
Annual out-of-sample stock-level prediction performance (percentage R^2_{oos})

	OLS +H	OLS-3 +H	PLS	PCR	ENet +H	GLM +H	RF	GBRT +H	NN1	NN2	NN3	NN4	NN5
All	–34.86	2.50	2.93	3.08	1.78	2.60	3.28	3.09	2.64	2.70	3.40	3.60	2.79
Top	–54.86	2.48	1.84	1.64	1.90	1.82	4.80	4.07	2.77	4.24	4.73	4.91	4.86
Bottom	–19.22	4.88	5.36	5.44	3.94	5.00	5.08	4.61	4.37	3.72	5.17	5.01	3.58

Annual return forecasting R^2_{oos} (see the legend to Table 1).

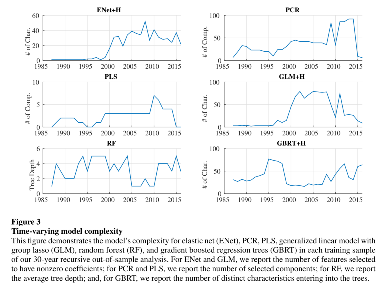


Figure 3
Time-varying model complexity
This figure demonstrates the model's complexity for elastic net (ENet), PCR, PLS, generalized linear model with group lasso (GLM), random forest (RF), and gradient boosted regression trees (GBRT) in each training sample of our 30-year recursive out-of-sample analysis. For ENet and GLM, we report the number of features selected to have nonzero coefficients; for PCR and PLS, we report the number of selected components; for RF, we report the average tree depth; and, for GBRT, we report the number of distinct characteristics entering into the trees.

Read:

- Table 1 is the headline table for individual-stock prediction. The strongest quantitative messages are:
  - OLS with all 920 features fails badly: $R^2_{oos} = -3.46\%$.
  - The benchmark OLS-3 model gets only about 0.16% monthly R^2.
  - Elastic net gets about 0.11%.
  - PCR and PLS raise this to about 0.27% and 0.26%.
  - GLM gets about 0.19%.
  - Random forest and GBRT reach about 0.33% and 0.34%.
  - Neural networks do best, peaking around 0.40% for NN3.
- Among **large stocks**, tree methods and neural networks are especially strong, with R^2_{oos} around **0.52%–0.70%**. This matters because it shows the gains are not just a tiny-stock illiquidity story.
- Table 2 shows that the same ranking of methods appears at the annual horizon, with annual R^2 values much larger than monthly ones.
- Figure 3 is useful because it shows how regularization actually works in practice:
  - elastic net often keeps only a small subset of predictors,
  - PCR uses around 20–40 components,
  - PLS uses very few components,
  - random forests stay shallow,
  - boosted trees use only a modest set of splitting variables.

Table 3
Comparison of monthly out-of-sample prediction using Diebold-Mariano tests

	OLS-3 +H	PLS	PCR	ENet +H	GLM +H	RF	GBRT +H	NN1	NN2	NN3	NN4	NN5
OLS+H	**3.26***	**3.29***	**3.35***	**3.29***	**3.28***	**3.29***	**3.26***	**3.34***	**3.40***	**3.38***	**3.37***	**3.38***
OLS-3+H		1.42	**1.87**	−0.27	0.62	**1.64**	1.28	1.25	**2.13**	**2.13**	**2.36**	**2.11**
PLS			−0.19	−1.18	−1.47	0.87	0.67	0.63	1.32	1.37	1.66	1.08
PCR				−1.10	−1.37	0.85	0.75	0.58	1.17	1.19	1.34	1.00
ENet+H					0.64	**1.90**	1.40	**1.73**	**1.97**	**2.07**	**1.98**	**1.85**
GLM+H						**1.76**	1.22	1.29	**2.28**	**2.17**	**2.68***	**2.37**
RF							0.07	−0.03	0.31	0.37	0.34	0.00
GBRT+H								−0.06	0.16	0.21	0.17	−0.04
NN1									0.56	0.59	0.45	0.04
NN2										0.32	−0.03	−0.88
NN3											−0.32	−0.92
NN4												−1.04

This table reports pairwise Diebold-Mariano test statistics comparing the out-of-sample stock-level prediction performance among thirteen models. Positive numbers indicate the column model outperforms the row model. Bold font indicates the difference is significant at 5% level or better for individual tests, and an asterisk indicates significance at the 5% level for 12-way comparisons via our conservative Bonferroni adjustment.

5.3 Table 3: statistical comparison across methods

Read:

- Table 3 uses modified Diebold–Mariano tests to compare forecast errors across methods.
- The first message is that **regularized linear models** significantly improve on unconstrained OLS.
- The second is that there is little statistical difference among the best constrained linear methods.
- The third is that **neural networks** are the only models that clearly and repeatedly dominate the linear and generalized linear benchmarks.
- Under conservative Bonferroni correction, the significance becomes weaker, but the performance ordering remains the same.

5.4 Figure 4, Figure 5, and Table 4: which predictors matter

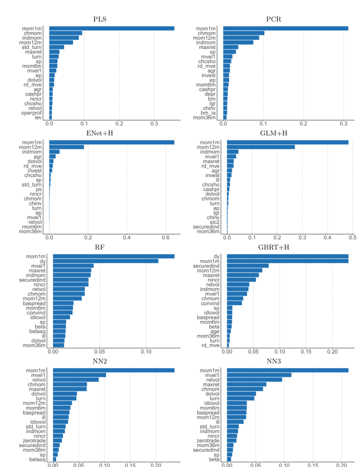


Figure 4
Variable importance by model
Variable importance for the top-20 most influential variables in each model. Variable importance is an average over all training samples. Variable importance within each model is normalized to sum to one.

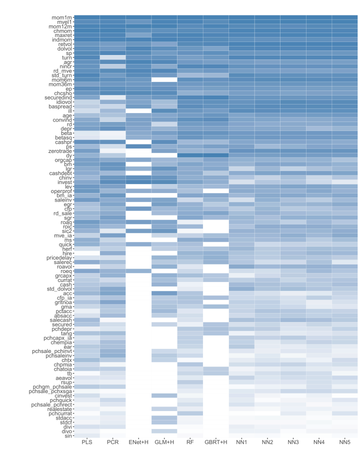
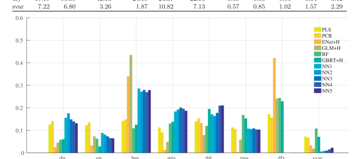


Figure 5
Characteristic importance
Rankings of ninety-four stock-level characteristics and the industry dummy (sic2) in terms of overall model contribution. Characteristics are ordered based on the sum of their ranks over all models, with the most influential characteristics on the top and the least influential on the bottom. Columns correspond to the individual models, and the color gradients within each column indicate the most influential (dark blue) to the least influential (white) variables.

Table 4
Variable importance for macroeconomic predictors

	PLS	PCR	ENet+H	GLM+H	RF	GBRT+H	NN1	NN2	NN3	NN4	NN5
dp	12.52	14.12	2.49	4.54	5.80	6.05	15.57	17.58	14.84	13.95	13.15
ep	12.25	13.52	3.27	7.37	6.27	2.85	8.86	8.09	7.34	6.54	6.47
bm	14.21	14.83	33.95	43.46	10.94	12.49	28.57	27.18	27.92	26.95	27.90
ntis	11.25	9.10	1.30	4.89	13.02	13.79	18.37	19.26	20.15	19.59	18.68
tbl	14.02	15.29	13.29	7.90	11.98	19.49	17.18	16.40	17.76	20.99	21.06
tms	11.35	10.66	0.31	5.87	16.81	15.27	10.79	10.59	10.91	10.38	10.33
dfy	17.17	15.68	42.13	24.10	24.37	22.93	0.09	0.06	0.06	0.04	0.12
svar	7.22	6.80	3.26	1.87	10.82	7.13	0.57	0.85	1.02	1.57	2.29



Variable importance for eight macroeconomic variables in each model. Variable importance is an average over all training samples. Variable importance within each model is normalized to sum to one. The lower panel provides a complementary visual comparison of macroeconomic variable importance.

Read:

- Figure 4 shows model-specific variable importance, while Figure 5 aggregates ranks across models.

- The most important predictors consistently fall into a few clusters:
  - **price trends:** short-term reversal, 12-month momentum, industry momentum, long-term reversal, recent max return,
  - **liquidity:** market equity, dollar volume, turnover, Amihud illiquidity, bid-ask spread,
  - **risk/volatility:** total volatility, idiosyncratic volatility, beta, beta squared,
  - **some valuation and fundamentals:** earnings-price, sales-price, asset growth, earnings-increase indicators.
- Penalized linear models put a lot of weight on momentum/reversal signals. Trees and neural nets spread importance over a broader set of signals.
- Table 4 shows that among macro variables, **aggregate book-to-market** is especially important. Linear methods emphasize bond-market variables like the Treasury-bill rate and default spread, while nonlinear methods place much more weight on variables like term spreads and issuance activity that linear models largely overlook.

5.5 Figure 6, Figure 7, and Figure 8: nonlinearities and interactions

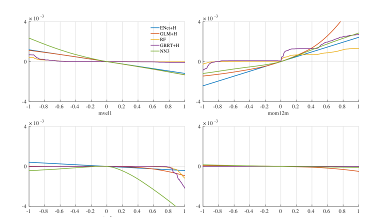


Figure 6
Marginal association between expected returns and characteristics
The panels should the sensitivity of expected monthly percentage returns (vertical axis) to the individual characteristics (holding all other covariates fixed at their median values).

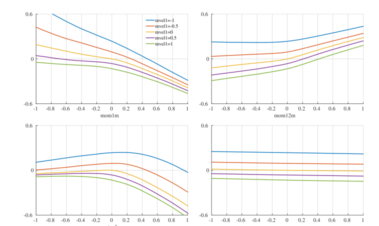


Figure 7
Expected returns and characteristic interactions (NN3)
The panels should the sensitivity of the expected monthly percentage returns (vertical axis) to the interactions effects for mvel1 with mom1m, mom12m, retvol, and acc in model NN3 (holding all other covariates fixed at their median values).

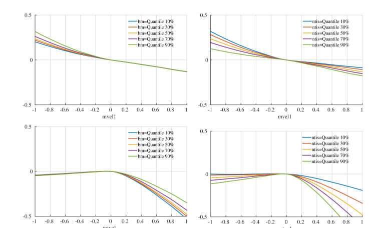


Figure 8
Expected returns and characteristic/macroeconomic variable interactions (NN3)
The panels show the sensitivity of expected monthly percentage returns (vertical axis) to interactions effects for mvel1 and retvol with bm and ntis in model NN3 (holding all other covariates fixed at their median values).

Read:

- Figure 6 is one of the most economically informative figures in the paper. It shows that machine-learning models recover familiar patterns:
  - expected returns decline with size,
  - increase with past 12-month returns,
  - and decline with volatility.
- But the figure also shows **why** nonlinear methods beat linear ones: the penalized linear model often finds almost no association for size or volatility, whereas NN3 and tree methods uncover strongly nonlinear effects.
- Quantitatively, NN3 implies that moving from median size to the 20th percentile raises annualized expected return by roughly **2.4%**, and moving from median volatility to the 80th percentile lowers annualized expected return by roughly **3.0%**.
- Figure 7 shows pairwise interactions in NN3. For example, the reversal effect is strongest among small stocks, while the momentum effect is more pronounced among larger stocks.
- Figure 8 shows interactions between firm characteristics and macro states. For example, the size effect becomes stronger when aggregate valuation is low or when net equity issuance is low.

5.6 Table 5 and Table 6: bottom-up portfolio forecasts and timing gains

Table 5
Monthly portfolio-level out-of-sample predictive R^2

	OLS-3 +H	PLS	PCR	ENet +H	GLM +H	RF	GBRT +H	NN1	NN2	NN3	NN4	NN5
A. Common factor portfolios												
S&P 500	−0.22	−0.86	−1.55	0.75	0.71	1.37	1.40	1.08	1.13	1.80	1.63	1.17
SMB	0.81	2.09	0.39	1.72	2.36	0.57	0.35	1.40	1.16	1.31	1.20	1.27
HML	0.66	0.50	1.21	0.46	0.84	0.98	0.21	1.22	1.31	1.06	1.25	1.24
RMW	−2.35	1.19	0.41	−1.07	−0.06	−0.54	−0.92	0.68	0.47	0.84	0.53	0.54
CMA	0.80	−0.44	0.03	−1.07	1.24	−0.11	−1.04	1.88	1.60	1.06	1.84	1.31
UMD	−0.90	−1.09	−0.47	0.47	−0.37	1.37	−0.25	−0.56	−0.26	0.19	0.27	0.35
B. Subcomponents of factor portfolios												
Big value	0.10	0.00	−0.33	0.25	0.59	1.31	1.06	0.85	0.87	1.46	1.21	0.99
Big growth	−0.33	−1.26	−1.62	0.70	0.51	1.32	1.19	1.00	1.10	1.50	1.24	1.11
Big neutral	−0.17	−1.09	−1.51	0.80	0.36	1.31	1.28	1.43	1.24	1.70	1.81	1.40
Small value	0.30	1.66	1.05	0.64	0.85	1.24	0.52	1.59	1.37	1.54	1.40	1.30
Small growth	−0.16	0.14	−0.18	−0.33	−0.12	0.71	1.24	0.05	0.42	0.48	0.41	0.50
Small neutral	−0.27	0.60	0.19	0.21	0.28	0.88	0.36	0.58	0.62	0.70	0.58	0.68
Big conservative	−0.57	−0.10	−1.06	1.02	0.46	1.11	0.55	1.15	1.13	1.59	1.37	1.07
Big aggressive	0.20	−0.80	−1.15	0.30	0.67	1.75	2.00	1.33	1.51	1.78	1.55	1.42
Big neutral	−0.29	−1.75	−1.96	0.83	0.48	1.13	0.77	0.85	0.85	1.51	1.45	1.16
Small conservative	−0.05	1.17	0.71	−0.02	0.34	0.96	0.56	0.82	0.87	0.96	0.90	0.83
Small aggressive	−0.10	0.51	0.01	−0.09	0.14	1.00	1.46	0.34	0.64	0.75	0.62	0.71
Small neutral	−0.30	0.45	0.12	0.42	0.35	0.76	−0.01	0.70	0.69	0.83	0.66	0.72
Big robust	−1.02	−1.08	−2.06	0.55	0.35	1.10	0.33	0.74	0.79	1.28	1.03	0.74
Big weak	−0.12	1.42	1.07	0.89	1.10	1.33	1.77	1.79	1.79	2.05	1.66	1.60
Big neutral	0.86	−1.22	−1.26	0.41	0.13	1.10	0.91	0.84	0.94	1.19	1.15	0.99
Small robust	−0.71	0.35	−0.38	−0.04	−0.42	0.70	0.19	0.24	0.50	0.63	0.53	0.55
Small weak	0.05	1.06	0.59	−0.13	0.44	1.05	1.42	0.71	0.92	0.99	0.90	0.89
Small neutral	−0.51	0.07	−0.47	−0.33	−0.32	0.60	−0.08	0.10	0.25	0.38	0.32	0.41
Big up	0.20	−0.25	−1.24	0.66	1.17	1.18	0.90	0.80	0.76	1.13	1.12	0.93
Big down	−1.54	−1.63	−1.55	0.44	−0.33	1.14	0.71	0.36	0.70	1.07	0.90	0.84
Big medium	−0.04	−1.51	−1.94	0.81	−0.08	1.57	1.80	1.29	1.32	1.71	1.55	1.23
Small up	0.07	0.78	0.56	−0.07	0.25	0.62	−0.03	0.06	0.07	0.21	0.19	0.25
Small down	−0.21	0.15	−0.20	0.15	−0.01	1.51	1.38	0.74	0.82	1.02	0.91	0.96
Small medium	0.07	0.82	0.20	0.59	0.37	1.22	1.06	1.09	1.09	1.18	1.00	1.03

In this table, we report the out-of-sample predictive R^2s for thirty portfolios using OLS with size, book-to-market, momentum, OLS-3, PLS, PCR, elastic net (ENet), generalized linear model with group lasso (GLM), random forest (RF), gradient boosted regression trees (GBRT), and five architectures of neural networks (NN1,...,NN5). "+H" indicates the use of Huber loss instead of the l_2 loss. The six portfolios in panel A are the S&P 500 index and the Fama-French SMB, HML, CMA, RMW, and UMD factors. The twenty-four portfolios in panel B are 3×2 size double-sorted portfolios used in the construction of the Fama-French value, investment, profitability, and momentum factors.

into large utility gains for a mean-variance investor. They show that the Sharpe ratio (SR^*) earned by an active investor exploiting predictive information (summarized as a predictive R^2) improves over the Sharpe ratio (SR) earned by a buy-and-hold investor according to

$$SR^* = \sqrt{\frac{SR^2 + R^2}{1 - R^2}}.$$

Table 6
Market timing Sharpe ratio gains

	OLS-3 +H	PLS	PCR	ENet +H	GLM +H	RF	GBRT +H	NN1	NN2	NN3	NN4	NN5
A. Common factor portfolios												
S&P 500	0.07	0.05	−0.06	0.12	0.19	0.18	0.19	0.22	0.20	0.26	0.22	0.19
SMB	0.06	0.17	0.09	0.24	0.26	0.00	−0.07	0.21	0.18	0.15	0.09	0.11
HML	0.00	0.01	0.04	−0.03	−0.02	0.04	0.02	0.04	0.06	0.04	0.02	0.01
RMW	0.00	−0.01	−0.06	−0.19	−0.13	−0.11	−0.01	−0.03	−0.09	0.01	0.01	−0.07
CMA	0.02	0.02	0	−0.09	−0.05	0.08	−0.01	0.00	0.01	0.05	0.04	0.06
UMD	0.01	−0.06	−0.02	−0.02	−0.07	−0.04	−0.07	−0.04	−0.08	−0.04	−0.10	−0.01
B. Subcomponents of factor portfolios												
Big value	−0.01	0.06	−0.03	0.09	0.06	0.09	0.08	0.11	0.11	0.13	0.10	0.11
Big growth	0.08	−0.01	−0.08	0.10	0.17	0.20	0.21	0.22	0.20	0.26	0.22	0.21
Big neutral	0.06	0.03	−0.06	0.11	0.16	0.13	0.17	0.23	0.21	0.23	0.23	0.21
Small value	−0.04	0.15	0.09	0.01	0.08	0.07	0.08	0.11	0.11	0.10	0.11	0.13
Small growth	0.00	0.03	−0.06	−0.03	−0.05	0.04	0.05	0.02	0.03	0.03	0.02	0.02
Small neutral	0.02	0.09	0.05	0.03	0.04	0.11	0.11	0.09	0.08	0.10	0.09	0.11
Big conservative	0.08	0.02	−0.04	0.08	0.15	0.09	0.13	0.17	0.14	0.19	0.16	0.14
Big aggressive	0.08	−0.01	−0.11	0.01	0.13	0.22	0.18	0.21	0.19	0.23	0.20	0.20
Big neutral	0.04	−0.01	−0.08	0.09	0.11	0.09	0.11	0.13	0.12	0.18	0.18	0.16
Small conservative	0.04	0.17	0.12	0.02	0.05	0.17	0.15	0.11	0.11	0.14	0.13	0.15
Small aggressive	0.01	0.05	−0.06	−0.05	−0.03	0.08	0.06	0.02	0.05	0.06	0.04	0.05
Small neutral	0.01	0.06	0.03	0.01	0.04	0.08	0.09	0.07	0.06	0.08	0.07	0.09
Big robust	0.10	0.07	−0.07	0.11	0.18	0.17	0.18	0.18	0.16	0.22	0.19	0.16
Big weak	0.05	0.12	0.05	0.09	0.12	0.21	0.17	0.22	0.20	0.21	0.18	0.19
Big neutral	0.09	0.00	−0.04	0.09	0.20	0.19	0.17	0.22	0.21	0.24	0.21	0.20
Small robust	0.09	0.04	−0.03	0.00	0.00	0.10	0.07	0.04	0.05	0.08	0.08	0.08
Small weak	−0.03	0.09	0.00	−0.03	−0.02	0.07	0.07	0.06	0.06	0.06	0.05	0.06
Small neutral	0.04	0.04	−0.03	0.00	0.01	0.11	0.09	0.04	0.04	0.07	0.07	0.08
Big up	0.10	0.05	−0.06	0.10	0.21	0.16	0.14	0.17	0.14	0.17	0.18	0.17
Big down	−0.02	0.09	−0.08	−0.02	0.02	0.08	0.10	0.10	0.07	0.12	0.11	0.09
Big medium	−0.01	0.04	−0.06	0.14	0.09	0.17	0.20	0.22	0.21	0.25	0.22	0.19
Small up	0.08	0.13	0.10	0.05	0.07	0.16	0.12	0.07	0.06	0.08	0.07	0.10
Small down	−0.14	0.04	−0.05	−0.09	−0.05	0.06	0.04	0.01	0.01	0.02	0.01	0.01
Small medium	0.05	0.11	0.07	0.08	0.09	0.13	0.15	0.13	0.12	0.14	0.13	0.15

This table documents improvement in annualized Sharpe ratio ($SR^* - SR$). We compute the SR^* by weighting the portfolios based on a market timing strategy (see Campbell and Thompson 2008).

Read:

- Table 5 shows that bottom-up portfolio prediction is where nonlinear methods really shine.
- For the **S&P 500**, the monthly out-of-sample R^2 values are roughly:
  - **-0.22%** for OLS-3,
  - **0.71%** for GLM,
  - **1.37%** for RF,
  - **1.40%** for GBRT,
  - and **1.80%** for NN3.
- NN3–NN5 produce positive R^2_{oos} for every portfolio the paper studies, including almost all characteristic-sorted long-only portfolios and most factor portfolios.
- Table 6 converts forecast power into market-timing Sharpe-ratio gains. The standout number is the S&P 500 result: timing with **NN3** raises the annualized Sharpe ratio by about **0.26**, from roughly **0.51** to about **0.77**.

- The gains are strongest for long-only broad portfolios. Long–short factor portfolios like UMD are much harder to time.

5.7 Table 7, Table 8, and Figure 9: machine-learning long–short portfolios

Table 7
Performance of the machine learning portfolios

	OLS-3+H				PLS				PCR			
	Pred	Avg	SD	SR	Pred	Avg	SD	SR	Pred	Avg	SD	SR
Low(L)	−0.17	0.40	5.90	0.24	−0.83	0.29	5.31	0.19	−0.68	0.03	5.98	0.02
2	0.17	0.58	4.65	0.43	−0.21	0.55	4.96	0.38	−0.11	0.42	5.25	0.28
3	0.35	0.60	4.43	0.47	0.12	0.64	4.63	0.48	0.19	0.53	4.94	0.37
4	0.49	0.71	4.32	0.57	0.38	0.78	4.30	0.63	0.42	0.68	4.64	0.51
5	0.62	0.79	4.57	0.60	0.61	0.77	4.53	0.59	0.62	0.81	4.66	0.60
6	0.75	0.92	5.03	0.63	0.84	0.88	4.78	0.64	0.81	0.81	4.58	0.61
7	0.88	0.85	5.18	0.57	1.06	0.92	4.89	0.65	1.01	0.87	4.72	0.64
8	1.02	0.86	5.29	0.56	1.32	0.92	5.14	0.62	1.23	1.01	4.77	0.73
9	1.21	1.18	5.47	0.75	1.66	1.15	5.24	0.76	1.52	1.20	4.88	0.86
High(H)	1.51	1.34	5.88	0.79	2.25	1.30	5.85	0.77	2.02	1.25	5.60	0.77
H-L	1.67	0.94	5.33	0.61	3.09	1.02	4.88	0.72	2.70	1.22	4.82	0.88

	ENet+H				GLM+H				RF			
	Pred	Avg	SD	SR	Pred	Avg	SD	SR	Pred	Avg	SD	SR
Low(L)	−0.04	0.24	5.44	0.15	−0.47	0.08	5.65	0.05	0.29	−0.09	6.00	−0.05
2	0.27	0.56	4.84	0.40	0.01	0.49	4.80	0.35	0.44	0.38	5.02	0.27
3	0.44	0.53	4.50	0.40	0.29	0.65	4.52	0.50	0.53	0.64	4.70	0.48
4	0.59	0.72	4.11	0.61	0.50	0.72	4.59	0.55	0.60	0.60	4.56	0.46
5	0.73	0.72	4.42	0.57	0.68	0.70	4.55	0.53	0.67	0.57	4.51	0.44
6	0.87	0.85	4.60	0.64	0.84	0.84	4.53	0.65	0.73	0.64	4.54	0.49
7	1.01	0.87	4.75	0.64	1.00	0.86	4.82	0.62	0.80	0.67	4.65	0.50
8	1.16	0.88	5.20	0.59	1.18	0.87	5.18	0.58	0.87	1.00	4.91	0.71
9	1.36	0.80	5.61	0.50	1.40	1.04	5.44	0.66	0.96	1.23	5.59	0.76
High(H)	1.66	0.84	6.76	0.43	1.81	1.14	6.33	0.62	1.12	1.53	7.27	0.73
H-L	1.70	0.60	5.37	0.39	2.27	1.06	4.79	0.76	0.83	1.62	5.75	0.98

	GBRT+H				NN1				NN2			
	Pred	Avg	SD	SR	Pred	Avg	SD	SR	Pred	Avg	SD	SR
Low(L)	−0.45	0.18	5.60	0.11	−0.38	−0.29	7.02	−0.14	−0.23	−0.54	7.83	−0.24
2	−0.16	0.49	4.93	0.35	0.16	0.41	5.89	0.24	0.21	0.36	6.08	0.20
3	0.02	0.59	4.75	0.43	0.44	0.51	5.07	0.35	0.44	0.65	5.07	0.44
4	0.17	0.63	4.68	0.46	0.64	0.70	4.56	0.53	0.59	0.73	4.53	0.56
5	0.34	0.57	4.70	0.42	0.80	0.77	4.37	0.61	0.72	0.81	4.38	0.64
6	0.46	0.77	4.48	0.59	0.95	0.78	4.39	0.62	0.84	0.84	4.51	0.65
7	0.59	0.52	4.73	0.38	1.11	0.81	4.40	0.64	0.97	0.95	4.61	0.71
8	0.72	0.72	4.92	0.51	1.31	0.75	4.86	0.54	1.13	0.93	5.09	0.63
9	0.88	0.99	5.19	0.66	1.58	0.96	5.22	0.64	1.37	1.04	5.69	0.63
High(H)	1.11	1.17	5.88	0.69	2.19	1.52	6.79	0.77	1.99	1.38	6.98	0.69
H-L	1.56	0.99	4.22	0.81	2.57	1.81	5.34	1.17	2.22	1.92	5.75	1.16

	NN3				NN4				NN5			
	Pred	Avg	SD	SR	Pred	Avg	SD	SR	Pred	Avg	SD	SR
Low(L)	−0.03	−0.43	7.73	−0.19	−0.12	−0.52	7.69	−0.23	−0.23	−0.51	7.69	−0.23
2	0.34	0.30	6.38	0.16	0.30	0.33	6.16	0.19	0.23	0.31	6.10	0.17
3	0.51	0.57	5.27	0.37	0.50	0.42	5.18	0.28	0.45	0.54	5.02	0.37
4	0.63	0.66	4.69	0.49	0.62	0.60	4.51	0.46	0.60	0.67	4.47	0.52
5	0.71	0.69	4.41	0.55	0.72	0.69	4.26	0.56	0.73	0.77	4.32	0.62
6	0.79	0.76	4.46	0.59	0.81	0.84	4.46	0.65	0.85	0.86	4.35	0.68
7	0.88	0.99	4.77	0.72	0.90	0.93	4.56	0.70	0.96	0.88	4.76	0.64
8	1.00	1.09	5.47	0.69	1.03	1.08	5.13	0.73	1.11	0.94	5.17	0.63
9	1.21	1.25	5.94	0.73	1.23	1.26	5.93	0.74	1.34	1.02	6.02	0.58
High(H)	1.83	1.69	7.29	0.80	1.89	1.75	7.51	0.81	1.99	1.46	7.40	0.68
H-L	1.86	2.12	6.13	1.20	2.01	2.26	5.80	1.35	2.22	1.97	5.93	1.15

In this table, we report the performance of prediction-sorted portfolios over the 30-year out-of-sample testing period. All stocks are sorted into deciles based on their predicted returns for the next month. Columns “Pred,” “Avg,” “SD,” and “SR” provide the predicted monthly returns for each decile, the average realized monthly

Table 8
Drawdowns, turnover, and risk-adjusted performance of machine learning portfolios

	OLS-3 +H	PLS	PCR	ENet +H	GLM +H	RF	GBRT +H	NN1	NN2	NN3	NN4	NN5
Drawdowns and turnover (value weighted)												
Max DD(%)	69.60	41.13	42.17	60.71	37.09	52.27	48.75	61.60	55.29	30.84	51.78	57.52
Max 1M loss(%)	24.72	27.40	18.38	27.40	15.61	26.21	21.83	18.59	37.02	30.84	33.03	38.95
Turnover(%)	58.20	110.87	125.86	151.59	145.26	133.87	143.53	121.02	122.46	123.50	126.81	125.37
Drawdowns and turnover (equally weighted)												
Max DD(%)	84.74	32.35	31.39	33.70	21.01	46.42	37.19	18.25	25.81	17.34	14.72	21.78
Max 1M loss(%)	37.94	32.35	22.33	32.35	15.74	34.63	22.34	12.79	25.81	12.50	9.01	21.78
Turnover(%)	57.24	104.47	118.07	142.78	137.97	120.29	134.24	112.35	112.43	113.76	114.17	114.34
Risk-adjusted performance (value weighted)												
Mean ret.	0.94	1.02	1.22	0.60	1.06	1.62	0.99	1.81	1.92	2.12	2.26	1.97
FF5+Mom α	0.39	0.24	0.62	−0.23	0.38	1.20	0.66	1.20	1.33	1.52	1.76	1.43
$t(\alpha)$	2.76	1.09	2.89	−0.89	1.68	3.95	3.11	4.68	4.74	4.92	6.00	4.71
R^2	78.60	34.95	39.11	28.04	30.78	13.43	20.68	27.67	25.81	20.84	20.47	18.23
IR	0.54	0.21	0.57	−0.17	0.33	0.77	0.61	0.92	0.93	0.96	1.18	0.92
Risk-adjusted performance (equally weighted)												
Mean ret.	1.34	2.08	2.45	2.11	2.31	2.38	2.14	2.91	3.31	3.27	3.33	3.09
FF5+Mom α	0.83	1.40	1.95	1.32	1.79	1.88	1.87	2.60	3.07	3.02	3.08	2.78
$t(\alpha)$	6.64	5.90	9.92	4.77	8.09	6.66	8.19	10.51	11.66	11.70	12.28	10.68
R^2	84.26	26.27	40.50	20.89	21.25	19.91	11.19	13.98	10.60	9.63	11.57	14.54
IR	1.30	1.15	1.94	0.93	1.58	1.30	1.60	2.06	2.28	2.29	2.40	2.09

The top two panels report value-weighted and equally weighted portfolio maximum drawdowns (“Max DD”), the most extreme negative monthly return (“Max 1M Loss”), and the average monthly percentage change in holdings (“Turnover”). The bottom panel reports average monthly returns expressed as a percentage as well as alphas, information ratios (IR), and R^2 with respect to the Fama-French five-factor model augmented to include the momentum factor.

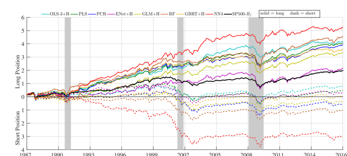


Figure 9
Cumulative return of machine learning portfolios
The figure shows the cumulative log returns of portfolios sorted on out-of-sample machine learning return forecasts. The solid and dashed lines represent long (top decile) and short (bottom decile) positions, respectively. The shaded periods show NBER recession dates. All portfolios are value weighted.

Read:

- Table 7 forms decile portfolios directly from model forecasts. The best value-weighted 10–1 spread comes from **NN4**:
  - average monthly return about **2.26%**,
  - monthly standard deviation about **5.80%**,
  - annualized Sharpe ratio about **1.35**.
- NN3 is almost as strong, with a value-weighted 10–1 spread average return around **2.12%** per month and Sharpe ratio around **1.20**.
- Equal-weighted long–short spreads are even stronger. The paper reports annualized Sharpe ratios up to about **2.45** for neural-network based strategies.
- Table 8 shows that neural-network portfolios also have attractive risk-adjusted performance:
  - NN4 has value-weighted FF5+momentum alpha around **1.76% per month** with a t-statistic around **6.00**,
  - and equal-weighted alpha around **3.08% per month** with a t-statistic around **12.28**.
- Neural networks also tend to have smaller drawdowns than the OLS benchmark, especially in equal-weight portfolios.
- Figure 9 shows cumulative returns of the machine-learning portfolios. NN4 clearly dominates the alternatives over the long sample.

- The meta-strategy that averages or rotates among machine-learning methods can raise stock-level predictive R^2 slightly further, but it does not dominate the best standalone neural network in Sharpe-ratio terms.

6 Short summary

- **Method.** The paper compares the canonical machine-learning methods for estimating conditional expected stock returns in a high-dimensional setting, using strict train/validation/test sample splitting and recursive out-of-sample forecasting.
- **Data.** The empirical application uses U.S. stocks from 1957–2016, with nearly 30,000 stocks, 94 firm characteristics, 8 macro variables, 74 industry dummies, and 920 baseline interacted features.
- **Main result.** Regularization and dimension reduction make large predictor sets usable, but the biggest gains come from nonlinear methods—especially regression trees and neural networks.
- **Economic meaning.** Neural-network forecasts materially improve market timing and long–short portfolio performance, and they imply that the most important predictive signals are price trends, liquidity, and volatility.

7 Conclusion

7.1 Core conclusion (what the paper establishes)

- The paper establishes that machine learning materially improves the measurement of expected stock returns relative to standard regression-based asset-pricing forecasts.
- Among the methods considered, **neural networks** and, to a lesser extent, **tree-based methods** perform best.
- The gain is not just statistical: it maps into large out-of-sample improvements in Sharpe ratios, alphas, and long–short trading performance.

7.2 Economic intuition

- The paper's central economic intuition is that expected returns are a complicated function of many state variables, and those state variables often matter through **interactions**.
- Linear regressions treat each predictor as having a constant marginal effect, which is too restrictive in a world where, for example, the effect of momentum may differ by firm size or macro state.
- Trees and neural networks can represent such state dependence cheaply, so they uncover predictive structure that linear models throw away.

7.3 Takeaway

This paper shows how to bring modern predictive methods into empirical asset pricing without abandoning economic discipline in evaluation. It does not solve the mechanism problem in finance, but it significantly improves the *measurement* problem. For anyone studying the cross-section or time series of expected returns, the message is clear: large predictor sets and nonlinear interactions contain economically meaningful forecasting information, and neural networks are especially effective at extracting it.